BUILDER SKILLS · PART 14

You Are Paying For The Same Tokens Twice.

Provider prompt caching and batch APIs cut LLM bills by 50 to 90 percent. Most teams never turn either one on. Here is exactly how they work.

system prompt	tools	few-shot	user turn
cached · ≈10% price	cached	cached	full price

16 slides · prefix caching · Anthropic vs OpenAI vs Gemini · batch jobs · a worked bill

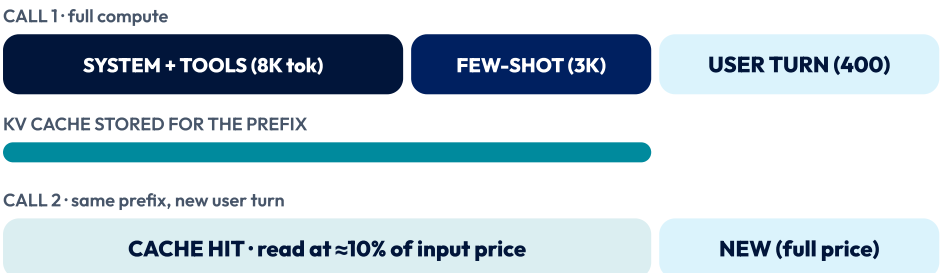
AI Coach John Follow

PROITBRIDGE
Strive For Better Future

Repost

What prompt caching actually reuses

Every call re-reads your whole prompt through the model. If the start of the prompt is identical to a recent call, the provider can reuse the attention state it already computed.



What is cached

The key/value tensors attention produced for the prefix tokens. Not the answer. The model still generates fresh output.

What you save

Input cost on the cached part drops to ≈10% (Anthropic, Gemini) or ≈50% (OpenAI). Time-to-first-token drops too, often by half.

What it is not

Not semantic caching. A single changed byte anywhere in the prefix means a miss. Exact prefix match only.

Who benefits most

RAG and agents: long system prompts, big tool schemas, many turns over the same document. Chatbots with 200-token prompts barely notice.

Anthropic vs OpenAI vs Gemini

Same idea, three different contracts. The differences decide how you structure prompts and what shows up on the invoice.

PROVIDER	HOW YOU OPT IN	CACHED READ	WRITE	TTL
Anthropic	Explicit cache_control breakpoints, up to 4 per request	≈10%	≈125% (5m) / ≈200% (1h)	5 min, or 1 h
OpenAI	Automatic on prompts ≥1,024 tokens, no code change	≈50%	none	≈5–10 min, up to 1 h off-peak
Gemini	Implicit (auto) plus explicit cache objects you create and reference	≈25%	storage per hour	You set it, default 1 h

MINIMUM CACHEABLE PREFIX



* *Anthropic's minimum is 1,024 tokens on Sonnet and Opus class models and 2,048 on Haiku. Below the minimum the breakpoint is silently ignored, no error.

Six things that silently break it

A cache miss costs the same as never caching, plus a write fee on Anthropic. These are the usual culprits.

01 **A timestamp in the system prompt.** "Today is 2026-09-26 14:03" changes every call. Move the date to the last user message.

02 **Tool order or tool schema drift.** Tools serialise into the prefix. Reordering a list in Python reorders the bytes.

03 **Per-user data at the top.** "The user's name is Priya" before the static instructions kills the shared prefix across users.

04 **Non-deterministic JSON.** Dict key order, float formatting, whitespace. Serialise once, reuse the string.

05 **Idle longer than the TTL.** A 5-minute cache and a cron that runs every 10 minutes never hits. Warm it or use the 1-hour tier.

06 **Changing model or parameters.** Caches are keyed per model. Switching sonnet → haiku for cheap turns starts cold.

identical prefix `1 byte` everything after the change is recomputed at full price

Static first, dynamic last

Caching works on prefixes, so ordering is the entire optimisation. Sort your prompt by how often each part changes.

1 · SYSTEM INSTRUCTIONS
`changes: per deploy · shared by every user`

2 · TOOL DEFINITIONS
`changes: per deploy · often 2–10K tokens`

3 · FEW-SHOT EXAMPLES / POLICY DOCS
`changes: weekly · biggest single win for RAG over a fixed corpus`

4 · CONVERSATION HISTORY
changes: every turn, but each turn extends the previous prefix → still cacheable

5 · CURRENT USER MESSAGE + RETRIEVED CHUNKS + DATE
changes: every call · never cached · keep it at the end

✓ Multi-turn trick

On Anthropic, put a breakpoint on the last user turn each call. The cache "rolls forward" and each turn only pays for the new tokens.

⚠ RAG trap

Retrieved chunks differ per query, so they cannot sit before the shared instructions. Chunks go after the cached block, always.

A cached system prompt in code

Anthropic needs one extra field per block. OpenAI needs only a stable prefix over 1,024 tokens.

```
# Anthropic: mark the end of the static block
import anthropic
client = anthropic.Anthropic()

resp = client.messages.create(
    model="claude-sonnet-5",
    max_tokens=800,
    system=[
        {"type": "text", "text": POLICY_DOC,        # ≈6K tokens, static
         "cache_control": {"type": "ephemeral"}},  # ← breakpoint
    ],
    tools=TOOLS,   # tools sit ABOVE system in the prefix
    messages=[{"role": "user", "content": question}],
)
u = resp.usage
print(u.cache_creation_input_tokens, u.cache_read_input_tokens, u.input_
# OpenAI: nothing to set → r.usage.prompt_tokens_details.cached_tokens
```

Anthropic prefix order is **tools → system → messages**. A breakpoint caches everything above it.

Log **cache_read / total input** per request. Under 60% on a long-prompt app means something is churning the prefix.

Reading the usage object

Caching is invisible unless you look at usage. Three numbers tell you whether it is working.

ONE REQUEST · 12,400 INPUT TOKENS · ANTHROPIC USAGE



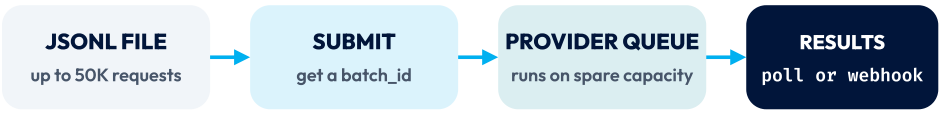
hit rate = 10,200 / 12,400 = 82% · effective input price ≈ 0.82×0.1 + 0.15×1.25 + 0.03×1 ≈ 0.30 of list
first call of a session shows write > 0 and read = 0. That is normal. Persistent write > 0 is the bug.

FIELD	PROVIDER	HEALTHY PATTERN
cache_read_input_tokens	Anthropic	Large and stable across calls in a session
cache_creation_input_tokens	Anthropic	Non-zero on the first call only
prompt_tokens_details.cached_tokens	OpenAI	Multiple of 128, close to your static prefix size
cached_content_token_count	Gemini	Equals the size of the referenced cache object

Build one dashboard panel: **cache hit rate by route**. When a deploy drops it from 80% to 5%, someone edited the system prompt or reordered tools.

Batch APIs: half price for patience

If nobody is waiting on the answer, do not pay for real-time. Submit a file of requests, get results back within 24 hours, at roughly half the price.



most batches finish in minutes to a few hours · the 24 h is a ceiling, not a typical wait

PROVIDER	DISCOUNT	WINDOW	LIMITS (APPROX.)
OpenAI Batch	≈50%	24 h	50K requests / file, separate rate pool
Anthropic Batches	≈50%	24 h	100K requests, stacks with caching
Gemini Batch	≈50%	24 h	File or inline, Vertex or AI Studio

Batch has its own rate limits. Teams use it as a second lane so back-office jobs never steal tokens-per-minute from the live product.

What belongs in a batch

The test is one question: does a human wait for this result? If no, it is batch work.

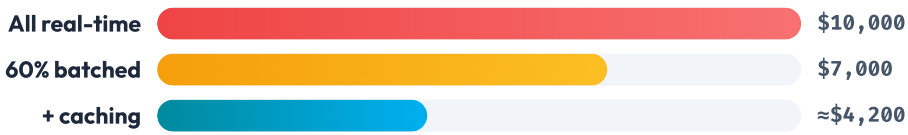
✓ Batch it

- Nightly evals over 5,000 golden questions
- Backfilling summaries for 2M support tickets
- Classifying, tagging, extracting fields at scale
- Generating synthetic training data
- Re-embedding a corpus after a model change
- LLM-as-judge scoring of yesterday's traffic

✕ Keep it live

- Chat and copilots, anything with a cursor blinking
- Agent loops where step N depends on step N-1
- Real-time moderation before a message posts
- Voice pipelines with latency budgets in ms
- Anything the user refreshes to see

A MONTHLY BILL, BEFORE AND AFTER



Pattern: one function, two call paths. **run_now()** for the product, **enqueue_batch()** for everything else. Same prompt, same tests.

Submitting a batch in 12 lines

Same shape on every provider: build requests with a custom id, submit, poll, join results by id.

```python
# OpenAI Batch: one JSONL line per request
import json, time
from openai import OpenAI
client = OpenAI()

with open("tickets.jsonl", "w") as f:
    for t in tickets:
        f.write(json.dumps({
            "custom_id": t.id,                    # join key
            "method": "POST", "url": "/v1/chat/completions",
            "body": {"model": "gpt-4.1-mini",
                     "messages": build_prompt(t)},
        }) + "\n")

up = client.files.create(file=open("tickets.jsonl", "rb"), purpose="batc
job = client.batches.create(input_file_id=up.id,
    endpoint="/v1/chat/completions", completion_window="24h")

while (job := client.batches.retrieve(job.id)).status not in ("completed
results = client.files.content(job.output_file_id).text   # JSONL, by cu
```

Individual lines can fail while the batch succeeds. Always read the **error_file_id** too.

Make requests idempotent by custom_id so a re-run only fills the gaps.

Caching + batch: the discounts stack

On Anthropic and Gemini the two are independent multipliers. A cached read inside a batch job is priced at both discounts.

LIST PRICE	× 0.5 BATCH	× 0.1 CACHE READ
$3.00 / M in	$1.50 / M	$0.15 / M

≈ 95% off the cached part of a batched prompt · Sonnet-class list prices, illustrative

order the batch file so requests sharing a prefix are adjacent · the cache warms once and stays warm

Nightly eval example

5,000 questions × 8K shared system prompt = 40M prefix tokens. Live and uncached: ≈$120. Batched and cached: ≈$6 plus the unique tokens.

Do not over-engineer

If a job costs $2 a night, leave it. Spend the engineering hour where the bill has four digits.

Rule: caching attacks the **repeated** tokens, batch attacks the **non-urgent** calls. Most spend is one, the other, or both.

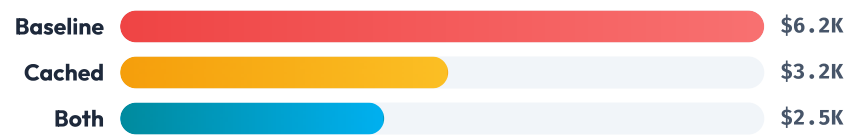
WORKED BILL

One RAG app, three ways to pay

A support assistant: 6K-token system prompt and tools, 2K retrieved chunks, 300-token question, 400-token answer. 200,000 calls per month.

8.3K	72%	200K
input tokens per call	of input is a static prefix	calls / month

SETUP	INPUT COST	OUTPUT COST	MONTHLY
No caching	1.66B tok × $3 = $4,980	80M × $15 = $1,200	≈$6,180
Prefix cached (80% hit)	≈$1,950	$1,200	≈$3,150
+ 40% traffic batched	≈$1,560	≈$960	≈$2,520



Output tokens are untouched by caching. When output dominates, shorten answers or use a smaller model. Caching cannot help there.

LONG DOCUMENTS

Caching a whole document once

"Chat with this 300-page PDF" is the textbook case. Load it into the cache once, then every question pays only for itself.

UPLOAD · 180K-token contract · cached for 1 hour — `paid once`

Q1 · 60 tokens	Q2 · 45 tokens	Q3 · 80 tokens	…×40

40 questions: uncached ≈7.2M input tokens · cached ≈0.9M-equivalent · ≈8× cheaper

PROVIDER	MECHANISM FOR LONG DOCS	WATCH OUT
Gemini	Create a CachedContent object, reference it by name in each call	Storage billed per token-hour, delete when done
Anthropic	Put the doc in a system block with a breakpoint, 1-hour TTL	≈2× write cost on first call, worth it past ≈3 questions
OpenAI	Keep the doc first in messages, identical bytes each call	Cache evicts after minutes of silence

⚖ **When this beats RAG: one document, one session, many questions. When RAG still wins: thousands of documents or answers that must cite a specific page.**

PITFALLS

Where teams get burned

Both features are cheap to turn on and easy to misuse. These are the recurring incidents.

1. **Paying cache writes with no reads.** Anthropic breakpoints on one-off prompts cost 25% extra. Only cache what repeats within the TTL.
2. **Stale instructions in a long-lived cache.** A Gemini cache with a 24 h TTL keeps serving yesterday's policy after you deploy. Version the cache name.
3. **Batch results nobody reads.** Output files expire. Persist results the moment the job completes, ideally via webhook.
4. **Assuming determinism.** A cache hit does not mean the same answer. Sampling still happens after the prefix.
5. **Privacy assumptions.** Prefix caches are scoped to your org, not shared across customers, but check the data-retention terms before caching regulated content.
6. **Optimising a prompt that is too short.** Under the 1,024-token floor nothing is cached at all. Measure before restructuring.

Test in staging: fire the same request twice, ten seconds apart, and assert the second shows cached tokens. Make it a CI check.

The 30-minute cost audit

Do this once per product. It usually finds a 40 to 70 percent cut with no change to model quality.

1. Pull one week of usage. Split input tokens into static prefix vs per-call.
2. Reorder prompts: instructions → tools → examples → history → user.
3. Remove timestamps, user names and request ids from the prefix.
4. Add breakpoints (Anthropic) or verify cached_tokens > 0 (OpenAI).
5. Tag every call site as **live** or **deferred**. Move deferred to batch.
6. Sort batch files by shared prefix so the cache warms once.
7. Dashboard: hit rate, write ratio, batch share, cost per 1K calls.
8. Alert when hit rate drops after a deploy.

≥1K	≈10%	≈50%	24h
tokens before caching helps	cached read price, Anthropic	batch discount, all three	batch completion ceiling

Caching and batching on one slide

CONCEPT	REMEMBER
Prefix caching	Provider reuses KV state for an identical prompt prefix. Exact bytes, in order.
Anthropic	Explicit cache_control, ≈10% read, ≈125% write, 5 min or 1 h TTL, ≥1,024 tokens.
OpenAI	Automatic, ≈50% read, no write fee, ≥1,024 tokens, minutes of TTL.
Gemini	Implicit plus explicit CachedContent objects, ≈25% read, storage per hour.
Layout	Static first, dynamic last. Tools → system → examples → history → user.
Cache killers	Timestamps, user data up top, tool reorder, unstable JSON, idle past TTL.
Metrics	cache_read / total input. Write should be non-zero only on first call.
Batch API	Async JSONL jobs, ≈50% off, 24 h ceiling, separate rate limits.
Batch fits	Evals, backfills, classification, synthetic data, embeddings, judge scoring.
Stacking	Batch × cache multiply. Sort batch files by shared prefix.
Limits	Output tokens never discounted by caching. Short prompts gain nothing.

⚡ **Repeated tokens → cache them. Non-urgent calls → batch them. Do both before you ever downgrade the model.**


AI Coach John Follow PROITBRIDGE Strive For Better Future Repost